M2S-03: Step 3 — Design: Create Target Architecture

> **Series:** M2S — Managed to SaaS Migration | **Notebook:** 3 of 9 | **Phase:** Plan | **Step:** Design | **Created:** March 2026 | **Last Updated:** 10/01/2026

With discovery and strategy complete, it's time to design the target architecture for your Dynatrace SaaS environment. This step produces the technical blueprints that guide every subsequent migration activity—network connectivity, ActiveGate topology, security controls, and high availability.

> **M2S Migration Journey — 3 Phases / 9 Steps**
>
> **Plan:** 1. Discover | 2. Strategize | **3. Design**
>
> **Upgrade:** 4. Prepare | 5. Execute | 6. Integrate
>
> **Run:** 7. Enable | 8. Expand | 9. Optimize

Table of Contents

1. Network Architecture
2. Network Zones
3. ActiveGate Design
4. Security Architecture
5. High Availability Design
6. Architecture Design Checklist

Prerequisites

Before starting this notebook, you should have:

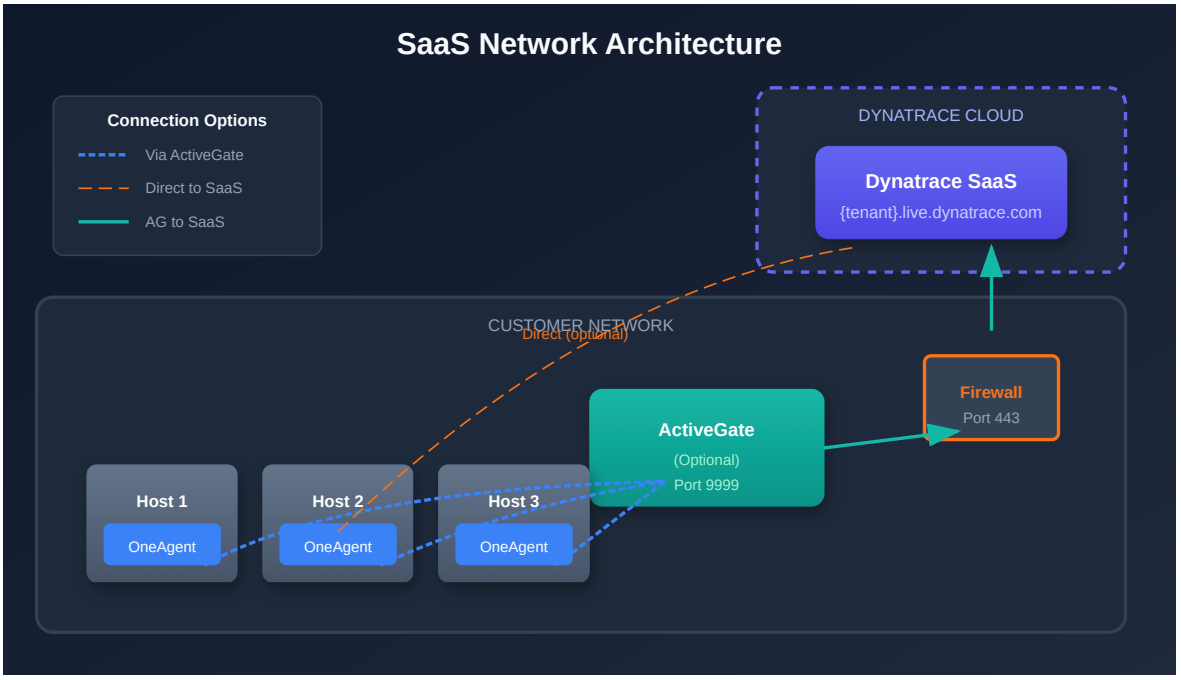
Requirement	Description
Steps 1–2 completed	Discovery inventory and migration strategy defined
SaaS tenant provisioned	Tenant ID and URL available
Network diagrams	Current Managed network topology documented
Firewall change process	Ability to request firewall rule changes
Security stakeholders engaged	Security and identity team involvement

> The Settings 2.0 schema for network zones (`builtin:networkzones.zones`) is catalogued alongside every other domain's schema in **AUTOM-02**'s consolidated Settings 2.0 Schema Catalog.

Learning Objectives

By the end of this notebook, you will:

- Design the network architecture for SaaS connectivity
- Plan network zone topology for optimized traffic routing
- Size and place ActiveGates for your environment
- Configure security controls including SSO and IAM
- Address high availability requirements
- Complete the architecture design checklist



---

1. Network Architecture

The most significant architectural change when moving to SaaS is connectivity—your monitored infrastructure must reach Dynatrace SaaS endpoints over the internet.

1.1 SaaS Endpoints

All communication uses outbound HTTPS (port 443).

Endpoint Pattern	Purpose
`{tenant-id}.live.dynatrace.com`	Main SaaS cluster (agent communication, API)
`{tenant-id}.apps.dynatrace.com`	Apps and platform services

1.2 Connectivity Options

Option	OneAgent →	ActiveGate →	Best For
Direct	SaaS (443)	N/A	Open internet access
Via Proxy	Proxy → SaaS	N/A	Proxy-controlled environments

Option	OneAgent →	ActiveGate →	Best For
Via ActiveGate	ActiveGate (9999)	SaaS (443)	Restricted networks
Via AG + Proxy	ActiveGate (9999)	Proxy → SaaS (443)	Most restricted environments

> **Recommendation:** For environments where hosts cannot reach the internet directly, deploy ActiveGates as routing proxies. OneAgents connect to the ActiveGate on port 9999; the ActiveGate forwards traffic to SaaS on port 443.

1.3 Firewall Requirements

Minimum Requirements (Direct Connectivity):

Source	Destination	Port	Protocol
OneAgent hosts	`{tenant-id}.live.dynatrace.com`	443	HTTPS
OneAgent hosts	`{tenant-id}.apps.dynatrace.com`	443	HTTPS

ActiveGate Routing (Restricted Networks):

Source	Destination	Port	Protocol
OneAgent hosts	ActiveGate	9999	HTTPS
ActiveGate	`{tenant-id}.live.dynatrace.com`	443	HTTPS
ActiveGate	`{tenant-id}.apps.dynatrace.com`	443	HTTPS

1.4 DNS Resolution

Ensure DNS can resolve these domains from all hosts and ActiveGates:

- `{tenant-id}.live.dynatrace.com`
- `{tenant-id}.apps.dynatrace.com`
- `*.dynatrace.com` (for additional services and updates)

1.5 Connectivity Testing

Test connectivity from representative hosts before migration:

```
# Test HTTPS connectivity to SaaS
curl -v https://{tenant-id}.live.dynatrace.com/api/v1/time

# Test apps endpoint
curl -v https://{tenant-id}.apps.dynatrace.com

# Test from behind ActiveGate (verify AG is reachable)
curl -v https://{activegate-host}:9999
```

> **Tip:** Run these tests from multiple network segments to identify connectivity gaps before migration begins.

1.6 Proxy Configuration

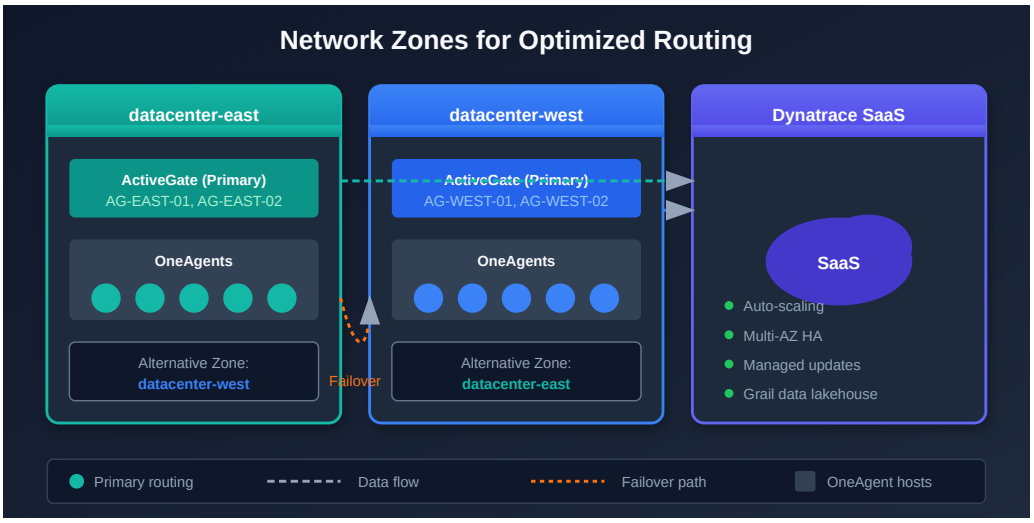
If a proxy is required for outbound HTTPS:

```
# OneAgent proxy configuration
sudo /opt/dynatrace/oneagent/agent/tools/oneagentctl --set-
proxy=http://proxy.example.com:8080

# ActiveGate proxy configuration — the [http.client] section of
# /var/lib/dynatrace/gateway/config/custom.properties:
#   [http.client]
#   proxy-server=proxy.example.com
#   proxy-port=8080
# ActiveGate 1.333+ can set the same keys with agctl:
sudo agctl property set --section=http.client --key=proxy-server --
value=proxy.example.com
sudo agctl property set --section=http.client --key=proxy-port --value=8080
# Restart the ActiveGate service afterwards.
```

> **Sources:** Proxy for ActiveGate (DT docs) — *"Specify the proxy-related parameters in the [http.client] section of the custom.properties file".*

Proxy Consideration	Detail
SSL inspection	Must not break TLS to Dynatrace endpoints
Authentication	Supported (basic auth in proxy URL)
Allowlist	`*.dynatrace.com` on port 443



Zone Failover

When no ActiveGate in an agent's own zone is reachable, OneAgents try the zone's configured **alternative zones**, then behave according to the zone's **fallback mode**:

Fallback mode	Behavior
Any ActiveGate (`ANY_ACTIVE_GATE` , default)	Traffic routes to any available ActiveGate in the environment
Only default zone (`ONLY_DEFAULT_ZONE`)	Traffic routes only to ActiveGates in the default zone
None (`NONE`)	Traffic is dropped rather than leaving the zone

Choose the mode deliberately: *Any ActiveGate* lets a zone failure land on ActiveGates sized for other zones, while *None* protects the fleet and drops data. FAQ-10 § 6 works

through the sizing consequences.

Assigning Zones at Scale

Zone membership is set per agent at install time (`--set-network-zone`) or afterwards with `oneagentctl --set-network-zone`. Drive it from the deployment automation you already use, keyed on the site, region or cloud account the host belongs to, so every new host lands in the right zone without a manual step.

> **Sources:** Get started with network zones (DT docs) — alternative zones and the three fallback modes; Network zones - Settings API (DT docs) — *"Valid values: ANY_ACTIVE_GATE (default), ONLY_DEFAULT_ZONE, NONE"*.

2. Network Zones

Network Zones optimize routing of telemetry data from OneAgents to ActiveGates to SaaS. They are especially important in multi-datacenter or hybrid-cloud environments.

2.1 Why Network Zones Matter

Benefit	Description
Traffic optimization	Keep traffic local within network segments
Failover routing	Automatic failover to alternative zones
Bandwidth reduction	Minimize cross-datacenter WAN traffic
Logical grouping	Group agents and AGs by physical topology

2.2 Network Zone Planning

Create zones that match your physical network topology:

Zone Type	Naming Example	Description
Datacenter-based	`datacenter-east`	Primary datacenter in East region

Zone Type	Naming Example	Description
Cloud region	`aws-us-east-1`	AWS region-specific zone
Environment-based	`production`	Production network segment
Hybrid	`dc-east-prod`	Datacenter + environment combined

2.3 Failover Configuration

Define alternative zones for each primary zone:

Primary Zone	Alternative Zone(s)	Failover Behavior
`datacenter-east`	`datacenter-west`	AGs in west serve east agents
`aws-us-east-1`	`aws-us-west-2`	Cross-region failover
`production`	`dr-production`	DR site serves production

2.4 Creating Network Zones

> **API 1.339 deprecation (May 2026):** the `/api/v2/networkZones` endpoints are deprecated — the reference page now reads *"This API is deprecated. Use the Settings API instead."* New automation should target the Settings 2.0 schema `builtin:networkzones.zones` via `POST /api/v2/settings/objects`. The legacy endpoint still works during the deprecation period; treat the Settings 2.0 form as canonical for new work.

Settings 2.0 (recommended for new automation):

```
curl -X POST "https://{tenant}.live.dynatrace.com/api/v2/settings/objects" \
  -H "Authorization: Api-Token {token}" \
  -H "Content-Type: application/json" \
  -d '[{
    "schemaId": "builtin:networkzones.zones",
    "scope": "environment",
    "value": {
      "id": "datacenter-east",
      "description": "Primary datacenter in East region",
      "alternativeZones": ["datacenter-west"],
      "fallbackMode": "ANY_ACTIVE_GATE"
    }
  }]'
```

`id`, `alternativeZones` (may be empty) and `fallbackMode` are required; `id` cannot be changed after creation. The token needs `settings.write`.

Legacy Configuration API (deprecated, still works during the deprecation window):

```
curl -X PUT
"https://{tenant}.live.dynatrace.com/api/v2/networkZones/datacenter-east" \
  -H "Authorization: Api-Token {token}" \
  -H "Content-Type: application/json" \
  -d '{
    "description": "Primary datacenter in East region",
    "alternativeZones": ["datacenter-west"]
  }'
# PUT creates the zone if it does not exist. Needs the networkZones.write
scope.
```

Assigning ActiveGates:

```
# During installation
sudo /bin/sh Dynatrace-ActiveGate-Linux.sh --set-network-zone=datacenter-east

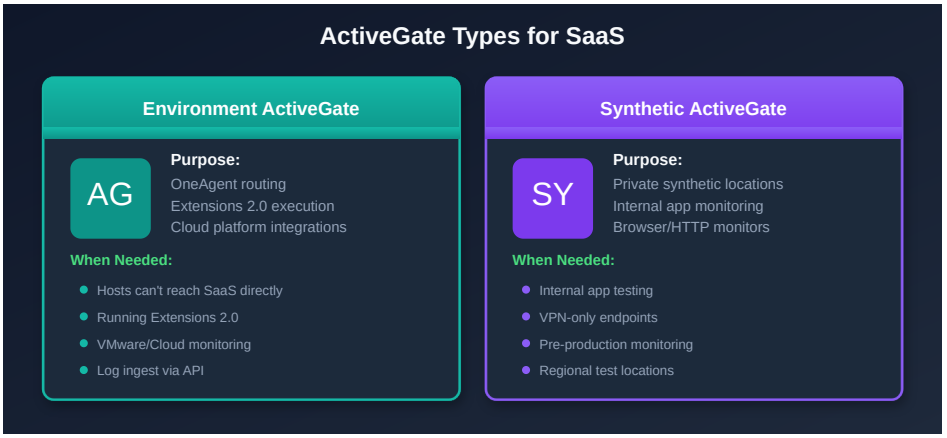
# After installation (ActiveGate 1.333+)
sudo agctl network-zone set datacenter-east
# Or in /var/lib/dynatrace/gateway/config/custom.properties:
#   [connectivity]
#   networkZone = datacenter-east
# Restart the ActiveGate service afterwards.
```

Assigning OneAgents:

```
# During installation
sudo /bin/sh Dynatrace-OneAgent-Linux.sh --set-network-zone=datacenter-east

# After installation
sudo /opt/dynatrace/oneagent/agent/tools/oneagentctl --set-network-
zone=datacenter-east
```

> **Important:** Every zone your agents use must exist in the SaaS tenant, with SaaS ActiveGates assigned, before you migrate OneAgents. The SaaS Upgrade Assistant lists network zones among the types it migrates; confirm they arrived and create any missing ones as part of Step 4 (Prepare).
>
> **Sources:** Network zones - Settings API (DT docs) — `id` : *"Unique identifier of the network zone"*; PUT a network zone (DT docs) — *"This API is deprecated. Use the Settings API instead."*; Configure ActiveGate (DT docs) — `[connectivity]` `networkZone` : *"Defines the network zone to which the ActiveGate belongs."*



---

3. ActiveGate Design

3.1 Do You Need ActiveGates?

Scenario	ActiveGate Required?
Hosts can reach SaaS directly	No (optional for extensions)
Hosts behind firewall, no direct internet	**Yes**, for routing
Running Extensions 2.0	**Yes** — a host-based ActiveGate for most remote extensions; SQL extensions can also run on Kubernetes through the Dynatrace Operator
Private synthetic monitoring	**Yes**
VMware monitoring	**Yes**

3.2 ActiveGate Sizing

Size from Dynatrace's published reference points, not a rule of thumb. The Linux requirements page gives estimated routed-host counts per instance size — for example roughly 800 hosts on a 2 vCPU / 3.75 GiB x86 instance, 1,800 on 4 vCPU / 7.5 GiB and 2,500 on 8 vCPU / 15 GiB, with ARM instances carrying more — and says the machine *"shouldn't exceed 50% CPU and 80% memory."* Extensions, log ingestion and synthetic workloads change the math. **FAQ-10** reproduces the full table, the log-ingestion sizing guide and the survivor-capacity rule for HA; use it rather than a host-count band.

> **Sources:** Linux ActiveGate hardware and system requirements (DT docs) — *"The machine running ActiveGate shouldn't exceed 50% CPU and 80% memory."*

3.3 Placement Best Practices

Practice	Rationale
Close to monitored hosts	Minimize latency between OneAgents and AGs

Practice	Rationale
Minimum 2 per network zone	High availability within each zone
Separate from monitored workloads	Avoid resource contention
In DMZ for cloud integrations	If needed for external access

3.4 ActiveGate Groups

Use AG groups to separate responsibilities:

Group Name	Purpose	Members
`production-routing`	Production OneAgent routing	AG-PROD-01, AG-PROD-02
`nonprod-routing`	Non-production routing	AG-NONPROD-01, AG-NONPROD-02
`extensions`	Extensions 2.0 execution	AG-EXT-01, AG-EXT-02
`synthetic`	Private synthetic locations	AG-SYNTH-01, AG-SYNTH-02

> **Note:** Plan host-based ActiveGates for Extensions 2.0. The one documented Kubernetes path is narrower: *"Run SQL monitoring extensions on Kubernetes using Dynatrace Operator"* (Extensions (DT docs)).

3.5 Parallel Deployment Strategy

Install new SaaS-connected ActiveGates in parallel with existing Managed ActiveGates:

1. **Deploy new AGs** connected to the SaaS tenant
2. **Validate connectivity** from AGs to SaaS endpoints
3. **Redirect OneAgents** to new AGs during migration
4. **Decommission Managed AGs** after all agents are migrated

This approach maintains monitoring continuity and provides easy rollback if issues occur.

3.6 ActiveGate Footprint for Cloud-Native and Serverless Estates

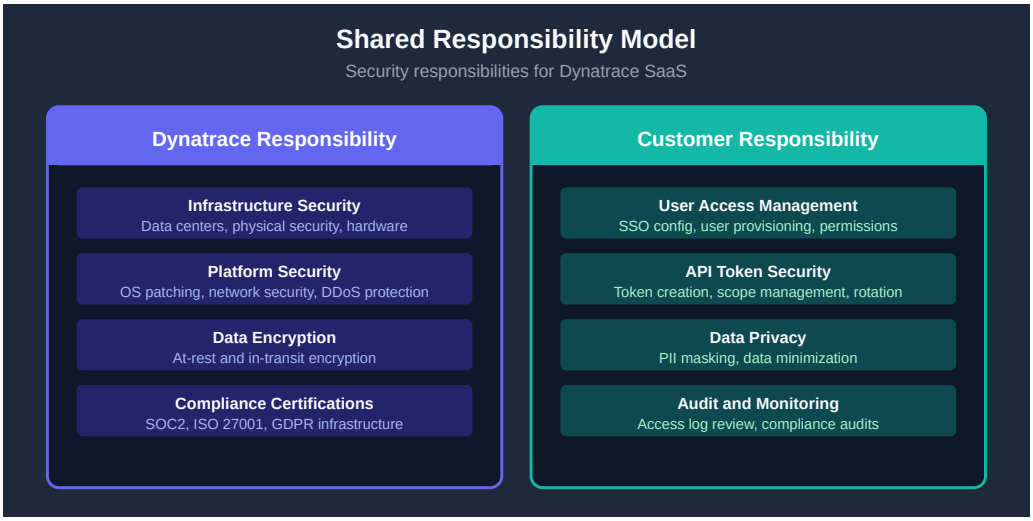
The sizing table above is keyed on **routed host count** — the right model for a VM-heavy estate. A Cloud Run / GKE / OTLP-heavy estate routes very little through ActiveGates, so its AG footprint is minimal and driven by specific needs rather than by total workload count:

Cloud-native need	ActiveGate involvement
Cloud Run / OTLP apps → SaaS	Typically none — telemetry goes to the SaaS ingest/OTLP endpoints directly (or via an existing routing AG only if the network requires it)
GKE OneAgent (DynaKube) → SaaS	Direct, or via an in-cluster ActiveGate the Dynatrace Operator can deploy — not sized from the host table
GCP log forwarding	Runs as a container workload (Pub/Sub + forwarder), **not** an ActiveGate — see **CLOUD-06: GCP Integration** for forwarder sizing
Private synthetic monitoring	Dedicated synthetic ActiveGate(s) — sized by synthetic load, not routed hosts
Residual VMs / VMware / Extensions 2.0	Host-based ActiveGate(s) sized normally for just those remaining hosts

> **Rule of thumb:** For a predominantly cloud-native estate, size ActiveGates for the *residual* host/VM count plus any private-synthetic and extension needs — not for the total workload count. Most Cloud Run and OTLP traffic never touches an ActiveGate.

```
// Inventory current ActiveGates and their versions
smartscapeNodes "ACTIVEGATE"
| fields name, version = dt.active_gate.version, zone = dt.network_zone.id
| sort name asc

// Correction (verified 07/2026): this cell previously ran `fetch
dt.entity.active_gate` and
// carried a note claiming Smartscape had no ActiveGate node. Both were wrong.
There is NO classic
// ActiveGate entity type in any spelling (active_gate,
environment_active_gate,
// environment_activegate), so the classic query returned zero rows in every
tenant —
// indistinguishable from "no ActiveGates deployed". `smartscapeNodes
"ACTIVEGATE"` (no
// underscore) is the working path, and it works on tenants today.
// Field maps: entity.name → name, softwareVersion → dt.active_gate.version,
// networkZone → dt.network_zone.id.
```



---

4. Security Architecture

4.1 SAML SSO Configuration

Dynatrace documents SAML 2.0 federation for SaaS user sign-in. If your Managed environment uses LDAP authentication, plan the move to SAML 2.0 through your identity provider.

Authentication	Managed	SaaS
Local users	Cluster Management Console	Dynatrace Account Management
SAML/SSO	IdP → Managed cluster	IdP → Dynatrace Account
LDAP	Direct LDAP integration	Not documented for SaaS — use SAML 2.0

> **Critical:** Your Identity Provider (IdP) must sign the **entire SAML message**, not just the assertion — Dynatrace answers an assertion-only signature with `400 Bad Request`. **Microsoft Entra ID does not do this by default:** its default for most gallery applications is *Sign SAML assertion*. Change the enterprise application's signing option to **Sign SAML response and assertion**.
>
> **Sources:** SAML (DT docs) — *"The entire SAML message must be signed (signing only SAML assertions is insufficient and generates a 400 Bad Request response)."*; Advanced certificate signing options in a SAML token (Microsoft Learn) — *"default option set for most of the gallery applications"*.

4.2 Azure Entra ID Considerations

Consideration	Requirement
SAML message signing	Set to **Sign SAML response and assertion** (the Entra default signs the assertion only)
Group claim limit	150 groups maximum per SAML token
Group filtering	**Filter claims to Dynatrace-related groups only**
Attribute mapping	Map UPN, email, first name, last name

> **Warning:** If a user belongs to more than 150 Azure Entra groups, the SAML token will contain a group overage claim instead of individual groups. Filter your SAML group claims to include only Dynatrace-relevant groups.

4.3 IAM Role Mapping

Map your existing Managed roles to SaaS IAM policies:

Managed Role	SaaS IAM Equivalent	Notes
Cluster administrator	Account administrator	Full account management
Environment administrator	Environment admin policy	Per-environment scope
Monitor user (read-only)	Viewer policy	Read-only access
Custom roles	Custom IAM policies	Recreate as policy statements

4.4 TLS and Encryption

Control	Detail
In transit	TLS for all agent, ActiveGate and API traffic
At rest	Encrypted by Dynatrace
Details	FAQ-24 § *stored* and the Dynatrace data-security-controls page cover the specifics; confirm current algorithms and key handling there rather than in this notebook

4.5 Data Residency

Decision	Impact
Region selection	Choose at provisioning time — AWS, Azure and Google Cloud regions are listed on the data-security-controls page
Treat as permanent	Moving regions means a new tenant and another migration
Compliance alignment	Select region matching your regulatory requirements

> **Important:** Data residency region is selected during SaaS provisioning. Treat it as permanent — moving a tenant's data to another region means provisioning a new

> tenant and migrating again. Confirm your region choice with compliance and legal teams before provisioning.

4.6 Firewall and Egress Controls

Control	Implementation
Egress filtering	Restrict outbound to `*.dynatrace.com` on port 443
IP allowlisting	SaaS endpoint IPs published by Dynatrace (subject to change)
DNS-based filtering	Preferred over IP-based (IPs may rotate)

4.7 API Token Strategy

Create purpose-specific tokens with minimal scopes:

Token Purpose	Required Scopes
OneAgent installation	`InstallerDownload`
ActiveGate installation	`InstallerDownload`
Configuration migration	`settings.read`, `settings.write`
Entity queries	`entities.read`
Metrics and logs	`metrics.read`, `logs.read`
AutomationEngine (Workflows)	OAuth client scopes `automation:workflows:read`, `automation:workflows:write` (not a classic API-token scope)

Bandwidth and Connectivity Planning

Consideration	Detail
Increased outbound bandwidth	Relocating the Dynatrace platform to SaaS increases outbound traffic from your network — plan for this with network team
SSL certificates	SSL certificates are possible for ActiveGates but NOT for the Dynatrace SaaS UI

Consideration	Detail
Service requests	Plan for change management procedures, change windows, and deployment freezes
OneAgent re-homing strategy	Choose one: (1) Uninstall agents and install new, or (2) Update agents in place via `oneagentctl` to keep IDs (host, process group, services) and metadata

5. High Availability Design

5.1 SaaS-Side HA (Provided by Dynatrace)

Dynatrace SaaS includes built-in high availability **within the region you choose** — data stays in that region:

Capability	Detail
Architecture	Clustered, across multiple availability zones, with automatic failover
Backups	Daily, to a separate account **in the same region** (AWS)
SLA	99.5% monthly uptime with Standard Support; 99.95% with Enterprise Success and Support

> **Sources:** Data security controls (DT docs) — *"performs data backups to a different AWS account in the same AWS region"*; Dynatrace SaaS SLA (Dynatrace).

5.2 Customer-Side HA

Your responsibility is ensuring no single points of failure in the agent-to-SaaS path:

Component	HA Strategy
ActiveGates	Minimum 2 per network zone, each able to carry the zone alone (FAQ-10 § 6)
Network zones	Define alternative zones and choose the fallback mode deliberately
Synthetic AGs	Deploy pairs at each private location

Component	HA Strategy
Load balancing	OneAgent discovers available ActiveGates itself — no load balancer in front of them

5.3 Failover Behavior

Scenario	OneAgent Behavior
Primary AG unavailable	Connects to another ActiveGate in the same zone
All AGs in zone unavailable	Tries the zone's alternative zones
No AG in the zone or its alternatives	Follows the zone's fallback mode (any ActiveGate, default zone only, or drop)
Connectivity restored	Resumes normal transmission

5.4 Design for Zero Single Points of Failure

Layer	Requirement
Network zones	At least one alternative zone per primary
ActiveGates per zone	Minimum 2 per zone
Network paths	Redundant paths from agents to AGs
DNS	Multiple DNS resolvers for SaaS domains

```
// Verify ActiveGate distribution across network zones
smartscapeNodes "ACTIVEGATE"
| summarize agCount = count(), by:{dt.network_zone.id}
| sort agCount desc

// Correction (verified 07/2026): this cell previously ran `fetch
dt.entity.active_gate` and
// carried a note claiming Smartscape had no ActiveGate node. Both were wrong.
There is NO classic
// ActiveGate entity type in any spelling (active_gate,
environment_active_gate,
// environment_activegate), so the classic query returned zero rows in every
tenant —
// indistinguishable from "no ActiveGates deployed". `smartscapeNodes
"ACTIVEGATE"` (no
// underscore) is the working path, and it works on tenants today.
// Field maps: entity.name → name, softwareVersion → dt.active_gate.version,
// networkZone → dt.network_zone.id.
```

6. Architecture Design Checklist

Complete this checklist before proceeding to Step 4 (Prepare).

Network

Checkpoint	Status
Network firewall rules documented (outbound 443 to SaaS)	[ ]
DNS resolution verified for `{tenant-id}.live.dynatrace.com`	[ ]
DNS resolution verified for `{tenant-id}.apps.dynatrace.com`	[ ]
Connectivity tested from representative hosts	[ ]
Proxy configuration documented (if required)	[ ]

Network Zones

Checkpoint	Status
Network zone topology designed (matching physical topology)	[ ]

Checkpoint	Status
Alternative zones defined for failover	[ ]
Zone naming convention established	[ ]

ActiveGates

Checkpoint	Status
ActiveGate sizing taken from the documented reference points (FAQ-10)	[ ]
Placement planned (close to monitored hosts)	[ ]
Minimum 2 AGs per network zone for HA	[ ]
AG groups defined (routing, extensions, synthetic)	[ ]
Host-based AGs planned for Extensions 2.0 (if needed)	[ ]

Security

Checkpoint	Status
SSO/SAML configuration designed	[ ]
IdP configured to sign full SAML message (Entra: *Sign SAML response and assertion*)	[ ]
Azure Entra group claims filtered to Dynatrace groups	[ ]
IAM policy mapping complete (Managed roles → SaaS policies)	[ ]
API token strategy defined with minimal scopes	[ ]
Data residency region confirmed with compliance team	[ ]
TLS 1.2+ confirmed for all connections	[ ]

High Availability

Checkpoint	Status
No single points of failure in AG topology	[ ]

Checkpoint	Status
Network zone failover paths verified	[ ]
SaaS SLA reviewed and documented	[ ]

Next Step

> → **M2S-04: Step 4 — Prepare** — Get your SaaS environment ready for migration execution. Deploy ActiveGates, configure network zones, create API tokens, and install the SaaS Upgrade Assistant.

Continue the Series

Next Notebook	Focus
M2S-04: Step 4 — Prepare	Environment setup and migration preparation

Architecture Resources

- ActiveGate Documentation
- Network Zones
- OneAgent Network Requirements
- IAM Documentation
- Dynatrace Trust Center
- SaaS Upgrade Assistant

Summary

In this notebook, you designed:

- **Network architecture** — Firewall rules, DNS, proxy configuration, and connectivity testing
- **Network zone topology** — Zones matching physical topology with failover alternatives
- **ActiveGate deployment** — Sizing, placement, groups, and parallel deployment strategy
- **Security architecture** — SAML SSO, IAM role mapping, TLS, data residency, and token strategy
- **High availability** — Redundant AGs, zone failover, and zero single points of failure

> **Key Takeaway:** The primary architectural change is network connectivity. Design your network zones and ActiveGate topology before migration begins—these decisions are difficult to change after OneAgents are redirected. Confirm data residency and SSO configuration with your compliance and security teams early.

---

*Continue to **M2S-04: Step 4 — Prepare** to set up your SaaS environment for migration execution.*

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.